



Examen final BookWorld



Contexte

BookWorld est une entreprise fictive spécialisée dans la vente de livres en ligne.

Elle souhaite mieux piloter son activité à l'international, mais ses données sont aujourd'hui dispersées entre plusieurs sources :

- un fichier CSV de ventes brutes,
- une base SQLite de référentiels métier,
- un site web à scraper pour récupérer les informations du catalogue (scraper seulement la première page),
- une API publique de taux de change pour transformer les données des prix scrapés des pounds en euros.

L'objectif du projet est de construire un pipeline de données permettant de :

- collecter les données depuis ces différentes sources ;
- produire un jeu de données final propre et cohérent des ventes par pays;
- créer une base SQL finale exploitable ;
- exposer via une API un indicateur métier simple : les ventes agrégées par pays.



Partie 1 : Créer le code source

1.1 Extraire les données depuis les différentes sources

Dans le fichier pipeline.py, mettre en œuvre l'extraction des données depuis :

- le fichier CSV : sales_raw.csv;
- la base SQLite : bookworld_reference.db ;
- la première page du site à scraper : <https://books.toscrape.com/> ;
- l'API publique :

Les attendus :

- lecture du CSV de ventes ;
- extraction des données de référence depuis SQLite ;
- scraping du catalogue de livres à partir de la première page du site books.toscrape.com ;
- récupération des taux de change entre les livres et l'euro via l'API publique Frankfurter ;
- gestion simple des erreurs et cas d'échec ;
- structuration du script avec un point d'entrée clair.

1.2. Écrire les requêtes SQL d'extraction

Produire un fichier queries.sql contenant les requêtes nécessaires à la collecte des données depuis la base SQLite.

Les attendus :

Le fichier doit contenir au minimum :

- une requête avec filtre ;
- une requête utile à l'enrichissement du pipeline.

Précision : les requêtes SQL peuvent d'abord être écrites dans `pipeline.py`, puis regroupées dans `queries.sql` à la fin. Il n'est pas obligatoire d'appeler dynamiquement le fichier `queries.sql` depuis Python. En revanche, `queries.sql` doit contenir la version finale lisible des requêtes utilisées dans le projet.



Partie 2 : Nettoyer, enrichir et agréger les données

2.1 À partir des données extraites, construire un jeu de données final cohérent.

Les attendus

Vous devez :

- A partir des données extraites, produire une agrégation sales_by_country.

Agrégation minimale attendue

L'agrégation sales_by_country doit contenir au minimum :

- country_code
- country_name
- total_orders
- total_quantity
- total_revenue_gbp (revenue en pounds)
- total_revenue_eur (revenue en euros)

Précision : le nettoyage, l'enrichissement et l'agrégation doivent rester simples et lisibles. L'objectif n'est pas de produire une analyse complexe, mais de montrer la capacité à construire un jeu de données final exploitable.

2.2 Modéliser la base finale

Une fois la base finale créée, produire un schéma de données qui décrit sa structure en utilisant un outil comme sqldesign: <https://sql.toad.cz/> ou miro, draw.io, ...

Les attendus :

- représenter les principales tables de la base finale ;
- représenter les relations utiles entre ces tables ;
- intégrer une capture d'écran du schéma dans le rapport final.



Partie 3 : Créer la base finale en prenant en compte le RGPD

3.1. Créer la base finale en prenant en compte le RGPD

Créer une base finale destinée à stocker les données du projet.

Les attendus :

- produire un fichier schema_final.sql ;
- créer une base finale alimentée par le pipeline ;
- exclure ou supprimer les colonnes nécessaires au regard du RGPD;
- justifier ce choix dans le rapport final au regard du RGPD.

Précision : le cas RGPD attendu dans ce projet est volontairement simple : les noms et prénoms sont présents dans les données brutes, mais ne sont pas nécessaires à l'usage final de la base. Ils ne doivent donc pas être conservés dans la base finale.



Partie 4 : Exposer les données via une API REST

Développer une API REST permettant de mettre à disposition les données finales stockées dans la base créée.

Les attendus

- produire un fichier api.py ;
- mettre en place une authentification simple par token ;
- documenter les endpoints dans le rapport final.

Endpoints minimum attendus :

- GET /health -> qui décrit si l'API fonctionne ou non
- GET /sales-by-country -> qui renvoie vers les données agrégées



Partie 5 : Documenter et publier le projet sur GitHub

Créer un fichier README.md et publier le projet dans un dépôt GitHub.

Les attendus

Le README.md doit expliquer :

- comment installer les dépendances ;
- comment exécuter le pipeline ;
- comment lancer l'API ;
- comment utiliser l'authentification par token/ clé SSH
- à quoi correspondent les principaux fichiers du projet.

Le projet complet doit être versionné et accessible via GitHub.



Partie 6 : Créer le rapport final

Produire un rapport final individuel présentant le projet et les choix réalisés.

Le rapport doit contenir :

- le contexte du projet ;
- les sources utilisées ;
- la logique d'extraction ;
- les requêtes SQL ;
- les choix de nettoyage et d'agrégation ;
- le schéma de la base finale ;
- le point RGPD – pourquoi avoir fait certains choix de nettoyage ;
- l'API et son authentification ;
- les limites et améliorations possibles.

Les attendus :

1 rapport au format docx :

- d'entre 2 et 5 pages (hors annexes)
- police Arial
- taille de police 12

